

אבטחת ווב: מהפרוטוקול ועד לניצול חולשות

Web Security: From the protocol to exploiting vulnerabilities



Workshop Labs:

https://github.com/m2a2/Research/tree/main/Workshops/2025_Hackeriot

מאור אבוטבול

אביתר גארזי

ליאור יקים

#Whoami / Whoweare



[Maor Abutbul]
<https://il.linkedin.com/in/maor-abutbul>



[Eviatar Gerzi]
<https://il.linkedin.com/in/eviatar-g-25bb6225>



[Lior Yakim]
<https://il.linkedin.com/in/lior-yakim-79b100156>



Agenda

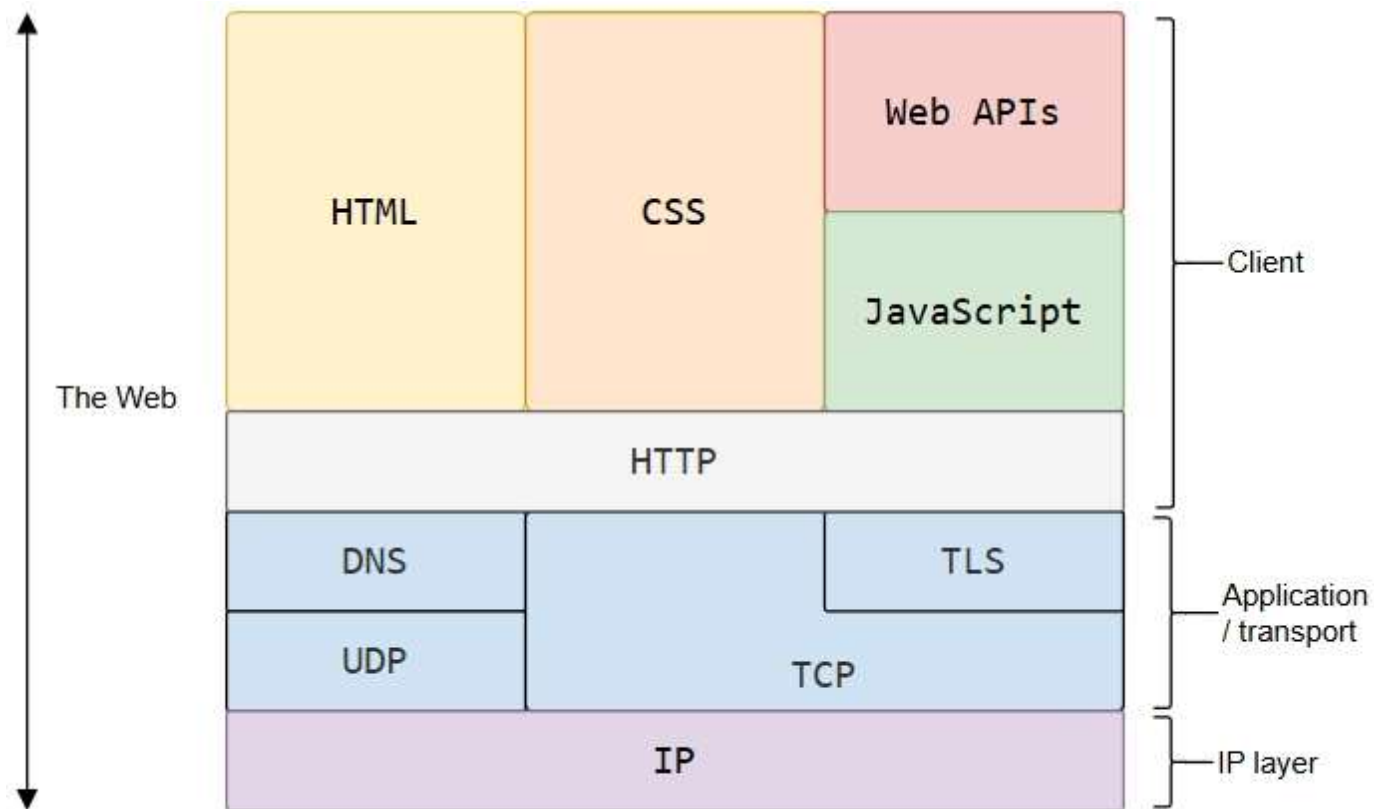
- רקע תיאורטי
- כלים לפיתוח ומחקר ווב
- אבטחת מערכות ווב – (Web Security)
 - חולשות נפוצות + מעבדה עצמית
- מקורות תרגול והעשרה + סיכום

טכנולוגיות ווב (בפיתוח) – Web Technologies

Web Technologies - HTML CSS JavaScript



Web Technologies / Layers



HTML - Hypertext Markup Language

HTML is the standard markup language for creating Web pages.

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>This is a Heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

This is a Heading

This is a paragraph.

<https://www.w3schools.com/html/default.asp>

JavaScript

- The world's most popular programming language.
- JavaScript is the programming language of the Web.



<https://www.w3schools.com/js/default.asp>

• הרצת קוד בצד לקוח (דפדפן)

Server-side and client-side technologies

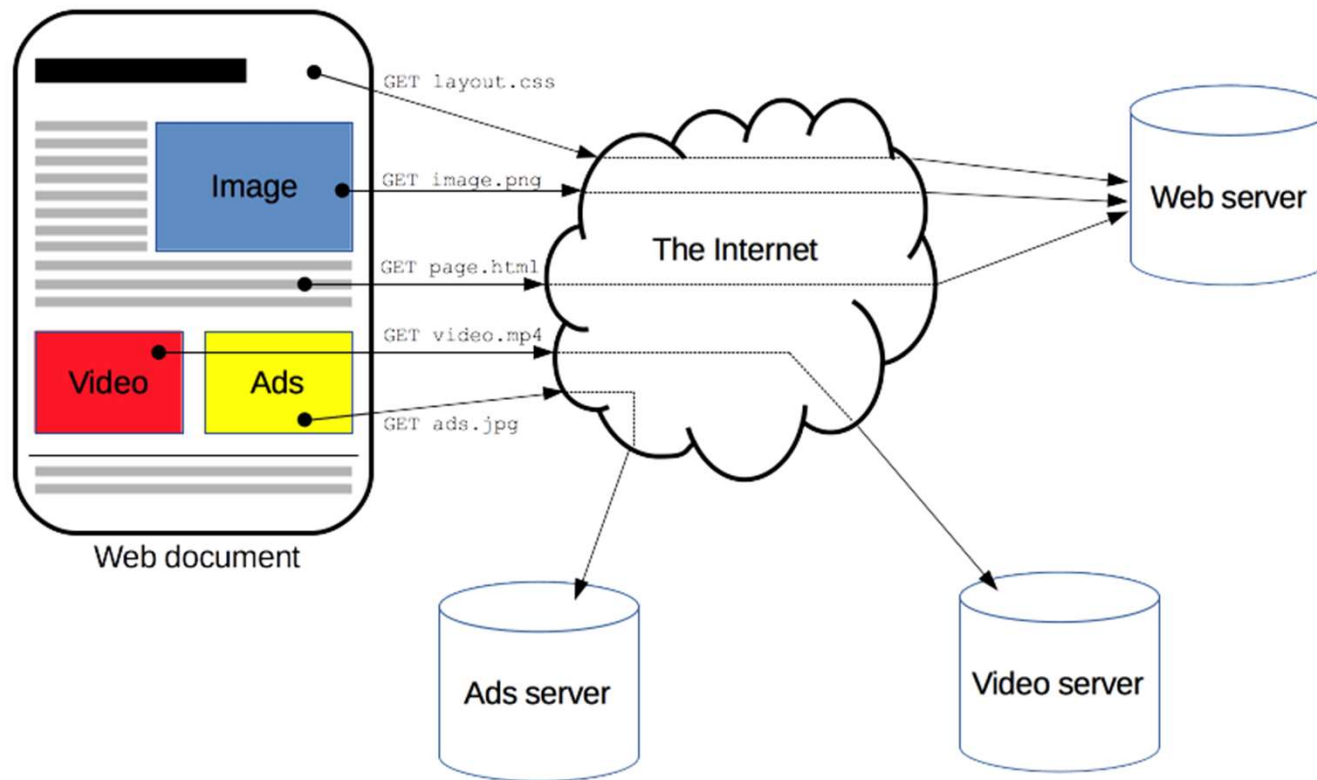
- Front-end development refers to the client-side (how a web page **looks**).
- Back-end development refers to the server-side (how a web page **works**).
- Server Side
 - PHP
 - ASP
- Web (Server) Frameworks
 - Node.js (JavaScript)
 - Django (Python)
- The request is being processed in the server-side.

HTTP Protocol

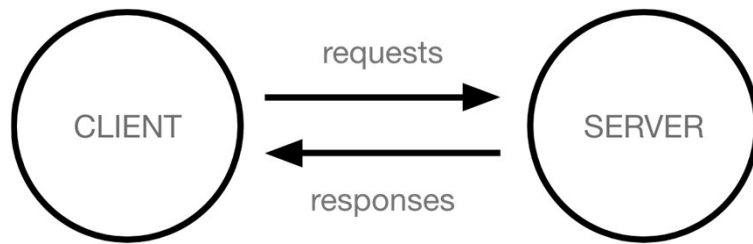
Hypertext Transfer Protocol (HTTP)



HTTP Protocol



HTTP Protocol



- מודל שרת-לקוח
- Stateless, but not session-less
- הודעות – בקשה תשובה/תגובה
- מידע נשלח כ-"טקסט" (ASCII)

<https://developer.mozilla.org/en-US/docs/Web/HTTP/>

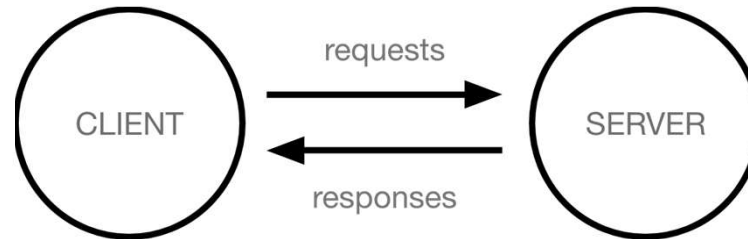
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview> -

HTTP Protocol

Method Path Protocol version



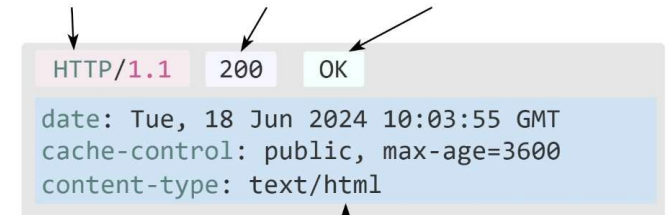
Headers



Responses are grouped in five classes:

1. Informational responses (100 – 199)
2. Successful responses (200 – 299)
3. Redirection messages (300 – 399)
4. Client error responses (400 – 499)
5. Server error responses (500 – 599)

Protocol version Status code Status message



Headers

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>

HTTP Protocol (in Wireshark)

The image shows a Wireshark packet capture of an HTTP GET request and response. The packet list on the left shows 13 packets. Packet 5 is the HTTP GET request, and packet 6 is the response. The packet details pane on the right shows the structure of the HTTP request and response.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	2a00:a041:e0f5:3a00::...	2600:1406:bc00:53::...	TCP	86	61438 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1440 WS=256 SACK_PERM
2	0.207057	192.168.1.244	23.220.75.245	TCP	66	61439 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3	0.223811	2600:1406:bc00:53::...	2a00:a041:e0f5:3a00::...	TCP	86	80 → 61438 [SYN, ACK] Seq=0 Ack=1 Win=64800 Len=0 MSS=1440 SACK_PERM WS=128
4	0.223955	2a00:a041:e0f5:3a00::...	2600:1406:bc00:53::...	TCP	74	61438 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
5	0.224248	2a00:a041:e0f5:3a00::...	2600:1406:bc00:53::...	HTTP	149	GET / HTTP/1.1
6	0.416984	23.220.75.245	192.168.1.244	TCP	66	80 → 61439 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
7	0.441104	2600:1406:bc00:53::...	2a00:a041:e0f5:3a00::...	TCP	74	80 → 61438 [ACK] Seq=1 Ack=76 Win=64768 Len=0
8	0.444887	2600:1406:bc00:53::...	2a00:a041:e0f5:3a00::...	TCP	1514	80 → 61438 [ACK] Seq=1 Ack=76 Win=64768 Len=1440 [TCP PDU reassembled in 9]
9	0.444887	2600:1406:bc00:53::...	2a00:a041:e0f5:3a00::...	HTTP	152	HTTP/1.1 200 OK (text/html)
10	0.445165	2a00:a041:e0f5:3a00::...	2600:1406:bc00:53::...	TCP	74	61438 → 80 [ACK] Seq=76 Ack=1519 Win=65280 Len=0
11	0.446471	2a00:a041:e0f5:3a00::...	2600:1406:bc00:53::...	TCP	74	61438 → 80 [FIN, ACK] Seq=76 Ack=1519 Win=65280 Len=0
12	0.683410	2600:1406:bc00:53::...	2a00:a041:e0f5:3a00::...	TCP	74	80 → 61438 [FIN, ACK] Seq=1519 Ack=77 Win=64768 Len=0
13	0.683608	2a00:a041:e0f5:3a00::...	2600:1406:bc00:53::...	TCP	74	61438 → 80 [ACK] Seq=77 Ack=1520 Win=65280 Len=0

Frame 5: 149 bytes on wire (1192 bits), 149 bytes captured (1192 bits) on interface
> Ethernet II, Src: Intel_cf:e5:fe (64:49:7d:cf:e5:fe), Dst: AlticeLabs_24:46:cf (68:00:27:24:46:cf)
> Internet Protocol Version 6, Src: 2a00:a041:e0f5:3a00::1983:58b1:a767:c892, Dst: 2600:1406:bc00:53::1
> Transmission Control Protocol, Src Port: 61438, Dst Port: 80, Seq: 1, Ack: 1, Len: 149
▼ Hypertext Transfer Protocol
 > GET / HTTP/1.1\r\n
 Host: example.com\r\n
 User-Agent: curl/8.14.1\r\n
 Accept: */*\r\n
 \r\n
 [Response in frame: 9]
 [Full request URI: http://example.com/]

0000 68 aa c4 24 46 cf 64 49 7d cf e5 fe 86 dd 60 0a h..\$.dI }.....
0010 bb 11 00 5f 06 40 2a 00 a0 41 e0 f5 3a 00 19 83 ..._@*..A:..
0020 58 b1 a7 67 c8 92 26 00 14 06 bc 00 00 53 00 00 X..g..&.....S..
0030 00 00 b8 1e 94 c8 ef fe 00 50 43 89 d7 eb 9a 31PC.....1
0040 3f aa 50 18 00 ff 0b 0d 00 00 47 45 54 20 2f 20 ?..P.....GET /
0050 48 54 54 50 2f 31 2e 31 0d 0a 48 6f 73 74 3a 20 HTTP/1.1 ..Host:
0060 65 78 61 6d 70 6c 65 2e 63 6f 6d 0d 0a 55 73 65 example..com..Use
0070 72 2d 41 67 65 6e 74 3a 20 63 75 72 6c 2f 38 2e r-Agent: curl/8.
0080 31 34 2e 31 0d 0a 41 63 63 65 70 74 3a 20 2a 2f 14.1..Ac cept: */
0090 2a 0d 0a 0d 0a *....

Text item (text), 16 bytes Packets: 15 · Dropped: 0 (0.0%) Profile: Classic

HTTP Protocol (in Wireshark)

No.	Time	Source	Destination	Protocol	Length	Info
5	0.224248	2a00:a041:e0f5:3a00...	2600:1406:bc00:53::...	HTTP	149	GET / HTTP/1.1
6	0.416984	23.220.75.245	192.168.1.244	TCP	66	80 → 61439 [SYN,
7	0.441104	2600:1406:bc00:53::...	2a00:a041:e0f5:3a00...	TCP	74	80 → 61438 [ACK]
8	0.444887	2600:1406:bc00:53::...	2a00:a041:e0f5:3a00...	TCP	1514	80 → 61438 [ACK]
9	0.444887	2600:1406:bc00:53::...	2a00:a041:e0f5:3a00...	HTTP	152	HTTP/1.1 200 OK
10	0.445165	2a00:a041:e0f5:3a00...	2600:1406:bc00:53::...	TCP	74	61438 → 80 [ACK]

> Frame 5: 149 bytes on wire (1192 bits), 149 bytes captured (1192 bits) on interface
> Ethernet II, Src: Intel_cf:e5:fe (64:49:7d:cf:e5:fe), Dst: AlticeLabs_24:46:cf (68
> Internet Protocol Version 6, Src: 2a00:a041:e0f5:3a00:1983:58b1:a767:c892, Dst: 260
> Transmission Control Protocol, Src Port: 61438, Dst Port: 80, Seq: 1, Ack: 1, Len:
✓ Hypertext Transfer Protocol
 > GET / HTTP/1.1\r\n
 Host: example.com\r\n
 User-Agent: curl/8.14.1\r\n
 Accept: */*\r\n
 \r\n
 [\[Response in frame: 9\]](#)
 [\[Full request URI: http://example.com/\]](#)

0000 68 aa
0010 bb 11
0020 58 b1
0030 00 00
0040 3f aa
0050 48 54
0060 65 78
0070 72 2d
0080 31 34
0090 2a 0d

אבטחת מערכות תקשורת - מבט מלמעלה

Network Security in 2 minutes



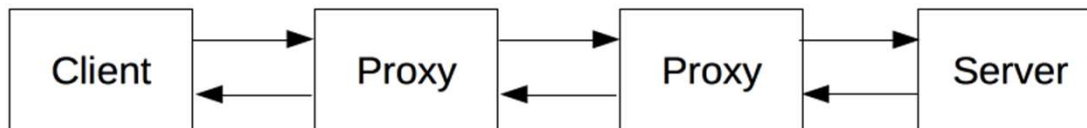
HTTP (דוגמאות ל) רכיבי תקשורת ואבטחה – עבור פרוטוקול HTTP

- רכיבי תקשורת (ואבטחה) ללא "נגיעה" HTTP (ניתוח)

- Firewall

- נתב (Router)

- Network Load Balancer



- רכיבים המנתחים תעבורת HTTP

- רכיב פרוקסי (Proxy)

- Application Load Balancer

- WAF – Web Application Firewall

- Web Server

<https://developer.mozilla.org/en-US/docs/Web/HTTP/>

כלים לפיתוח ומחקר ווב

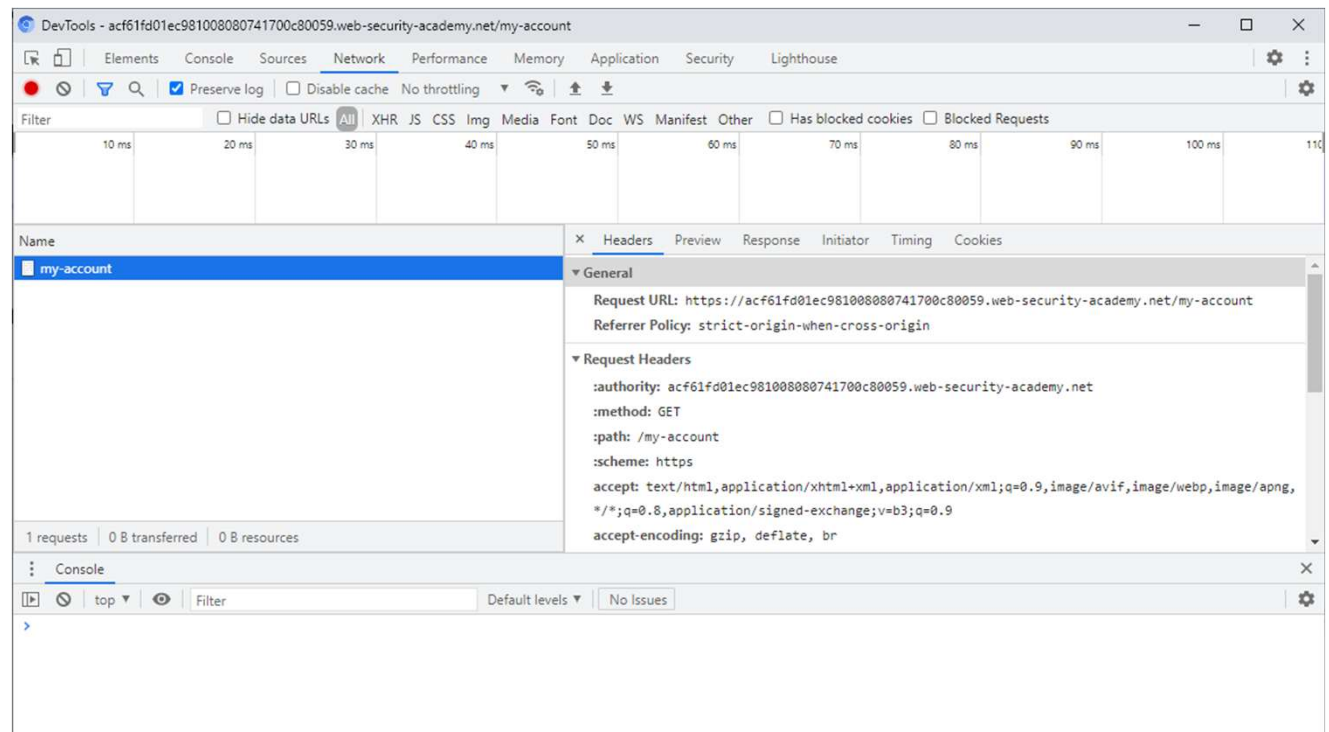
Tools & Demos

Agenda

- רקע תיאורטי
- **כלים לפיתוח ומחקר ווב**
- אבטחת מערכות ווב – (Web Security)
 - חולשות נפוצות + מעבדה עצמית
 - מקורות תרגול והעשרה + סיכום

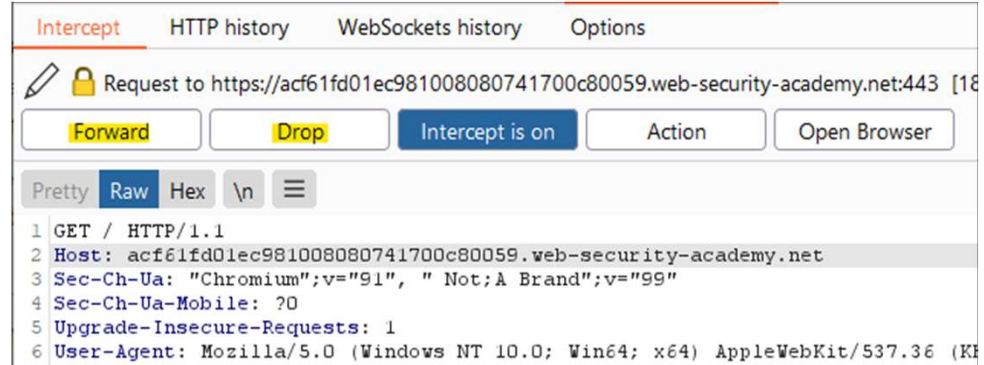
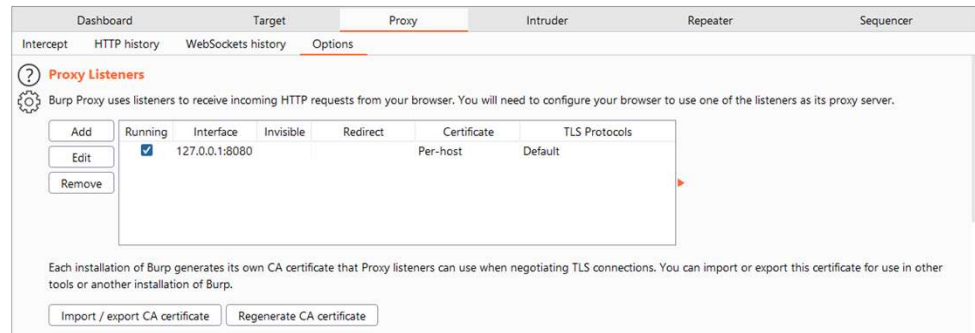
Browser DevTools + Demo

- Elements
- Console
 - Page interaction
- Application
 - Cookies
- Sources
 - Debug
- Network
 - Copy As
 - (fetch + console)



Tools - Burp Suite (Community)

- Host Proxy
- Dashboard
- Target
- Proxy
 - Intercept
 - HTTP History
 - Options



<https://portswigger.net/burp/communitydownload>

Burp Suite Cont'

- Repeater (Ctrl + R)

• עריכה ושליחה ידנית

- Intruder (Ctrl + I)

• הגדרת "משתנים" וערכי Payload
• לשליחה אוטומטית

- Decoder

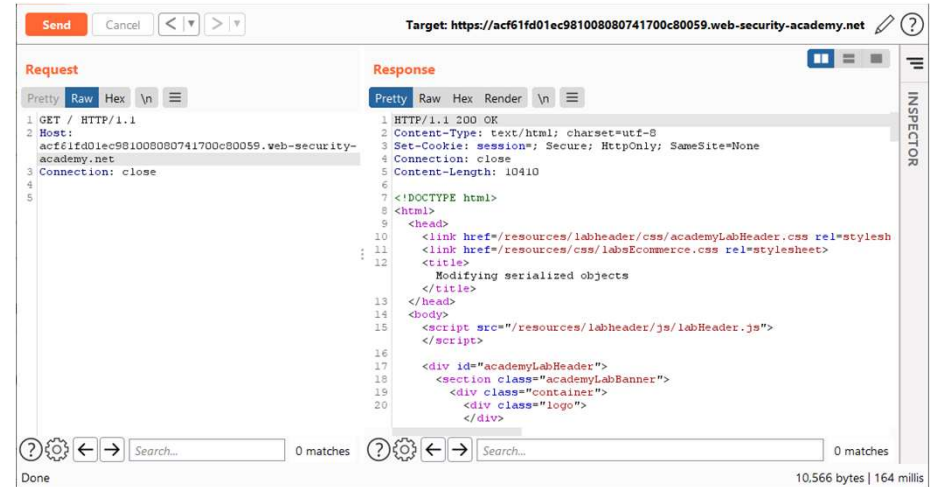
• פענוח קידודים

- Comparer

• השוואה בין בקשות

- Extender

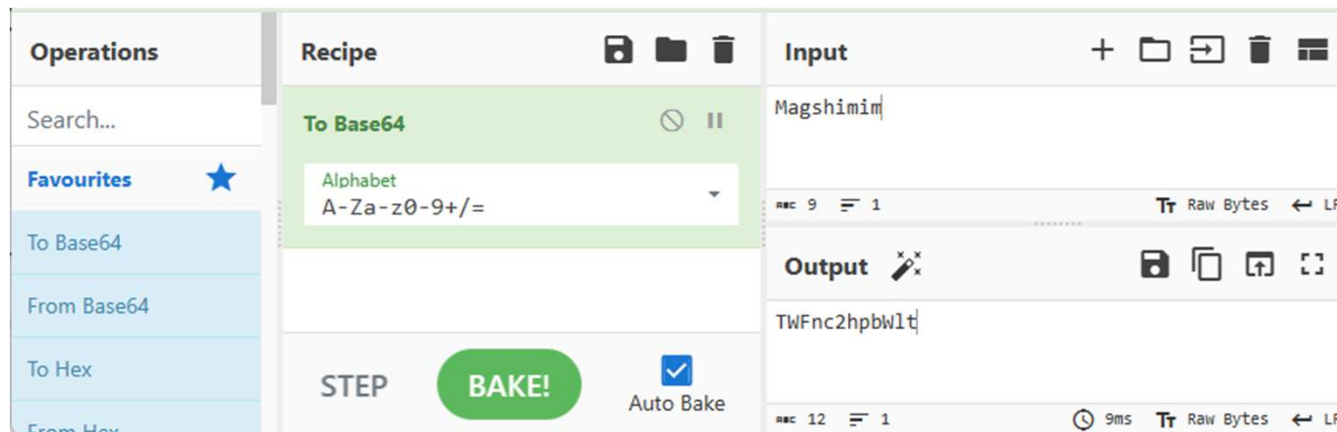
• הורדת וניהול "הרחבות"



Base64 (&URL) Encoding + CyberChef

- קידוד - Base64

- מיועד להעברת מידע בינארי על גבי ערוץ שתומך בטקסט בלבד (תווי אסקי)
- משמש לשליחת צרופות (קבצים) בשליחת מיילים (SMTP)



- <https://en.wikipedia.org/wiki/Base64>
- [https://gchq.github.io/CyberChef/#recipe=To_Base64\('A-Za-z0-9%2B/%3D'\)From_Base64\('A-Za-z0-9%2B/%3D',true,false/disabled\)&input=SGFja2VyaW90](https://gchq.github.io/CyberChef/#recipe=To_Base64('A-Za-z0-9%2B/%3D')From_Base64('A-Za-z0-9%2B/%3D',true,false/disabled)&input=SGFja2VyaW90)

Tools

- Browser Developer Tools (Dev-Tools F12)
- (Host) Proxy Tools
 - Burp Suite - <https://portswigger.net/burp/communitydownload>
 - ZAP Proxy - <https://www.zaproxy.org/>
 - Fiddler - <https://www.telerik.com/fiddler/fiddler-classic>
- Network Analyzer (Wireshark)
 - <https://www.wireshark.org/download.html>
- CyberChef (Data manipulations / transformations)
 - <https://gchq.github.io/CyberChef/>

Web Security

Resources - מקורות



Agenda

- רקע תיאורטי
- כלים לפיתוח ומחקר ווב
- **אבטחת מערכות ווב – (Web Security)**
 - חולשות נפוצות + מעבדה עצמית
 - Path Traversal
 - Access Control
 - Authentication
 - Insecure deserialization
- מקורות תרגול והעשרה + סיכום

רגע לפני – אחריות – חוק המחשבים

יש חוק בישראל

- חוק המחשבים, התשנ"ה–1995

- [חוק המחשבים - קישור](#)

- הסבר מתוך קורס של המרכז לחינוך סייבר

- [חוק המחשבים.pdf](#)

- ובכל מקרה

- חובה להשיג אישור (בכתב!) של בעל הנכס.

- לשים לב שלא חורגים מהנכסים שהוגדרו לנו.

- לדוגמא:

- שרתים משותפים.

- שירותי אחסון.

Web Security - Resources - מקורות

OWASP - Open Web Application Security Project

- <https://owasp.org/>

OWASP Top Ten

- <https://owasp.org/Top10/>

OWASP Web Security Testing Guide

- <https://owasp.org/www-project-web-security-testing-guide/v42/>

Portswigger - Web Security Academy

- <https://portswigger.net/web-security/learning-path>

Web Security – Resources - Portswigger - מקורות

Client-side topics

Client-side vulnerabilities introduce an additional layer of complexity, which can make them slightly more challenging. These materials and labs will help you build on the server-side skills you've already learned and teach you how to identify and exploit some gnarly client-side vectors as well.

Cross-site scripting (XSS)

Simply put, XSS is one of the most important vulnerabilities out there. It's both incredibly common and extremely powerful, especially when used as part of a wider exploit chain. This is a huge topic, with plenty of labs for complete beginners and seasoned pros alike.

[Go to topic →](#) 30 Labs

Cross-site request forgery (CSRF)

[Go to topic →](#) 12 Labs

Cross-origin resource sharing (CORS)

[Go to topic →](#) 4 Labs

Clickjacking

[Go to topic →](#) 5 Labs

DOM-based vulnerabilities

[Go to topic →](#) 7 Labs

WebSockets

[Go to topic →](#) 3 Labs

Server-side topics

For complete beginners, we recommend starting with our server-side topics. These vulnerabilities are typically easier to learn because you only need to understand what's happening on the server. Our materials and labs will help you develop some of the core knowledge and skills that you will rely on time after time.

SQL injection

SQL injection is an old-but-gold vulnerability responsible for many high-profile data breaches. Although relatively simple to learn, it can potentially be used for some high-severity exploits. This makes it an ideal first topic for beginners, and essential knowledge even for more experienced users.

[Go to topic →](#) 15 Labs

Authentication

[Go to topic →](#) 14 Labs

Path traversal

[Go to topic →](#) 6 Labs

Command injection

[Go to topic →](#) 5 Labs

Business logic vulnerabilities

[Go to topic →](#) 11 Labs

Information disclosure

[Go to topic →](#) 5 Labs

Access control

[Go to topic →](#) 13 Labs

File upload vulnerabilities

[Go to topic →](#) 7 Labs

Race conditions

[Go to topic →](#) 6 Labs

Server-side request forgery (SSRF)

[Go to topic →](#) 7 Labs

XXE injection

[Go to topic →](#) 9 Labs

NoSQL injection

[Go to topic →](#) 4 Labs

API testing

[Go to topic →](#) 5 Labs

PortSwigger - Web Security Academy

<https://portswigger.net/web-security/all-topics>

Web Security – Resources - Portswigger - מקורות

Advanced topics

These topics aren't necessarily more difficult to master but they generally require deeper understanding and a wider breadth of knowledge. We recommend getting to grips with the basics before tackling these labs, some of which are based on pioneering techniques discovered by our world-class research team.

Insecure deserialization

Deserialization has a reputation for being difficult to get your head around but it can be much easier to exploit than you might think. We'll guide you through the process step-by-step so you can pick off some high-severity bugs that even experienced testers may have missed altogether.

[Go to topic →](#)

10 Labs

Web LLM attacks

New topic

[Go to topic →](#)

4 Labs

GraphQL API vulnerabilities

[Go to topic →](#)

5 Labs

Server-side template injection

[Go to topic →](#)

7 Labs

Web cache poisoning

[Go to topic →](#)

13 Labs

HTTP Host header attacks

[Go to topic →](#)

7 Labs

HTTP request smuggling

[Go to topic →](#)

22 Labs

OAuth authentication

[Go to topic →](#)

6 Labs

JWT attacks

[Go to topic →](#)

8 Labs

Prototype pollution

[Go to topic →](#)

10 Labs

Essential skills

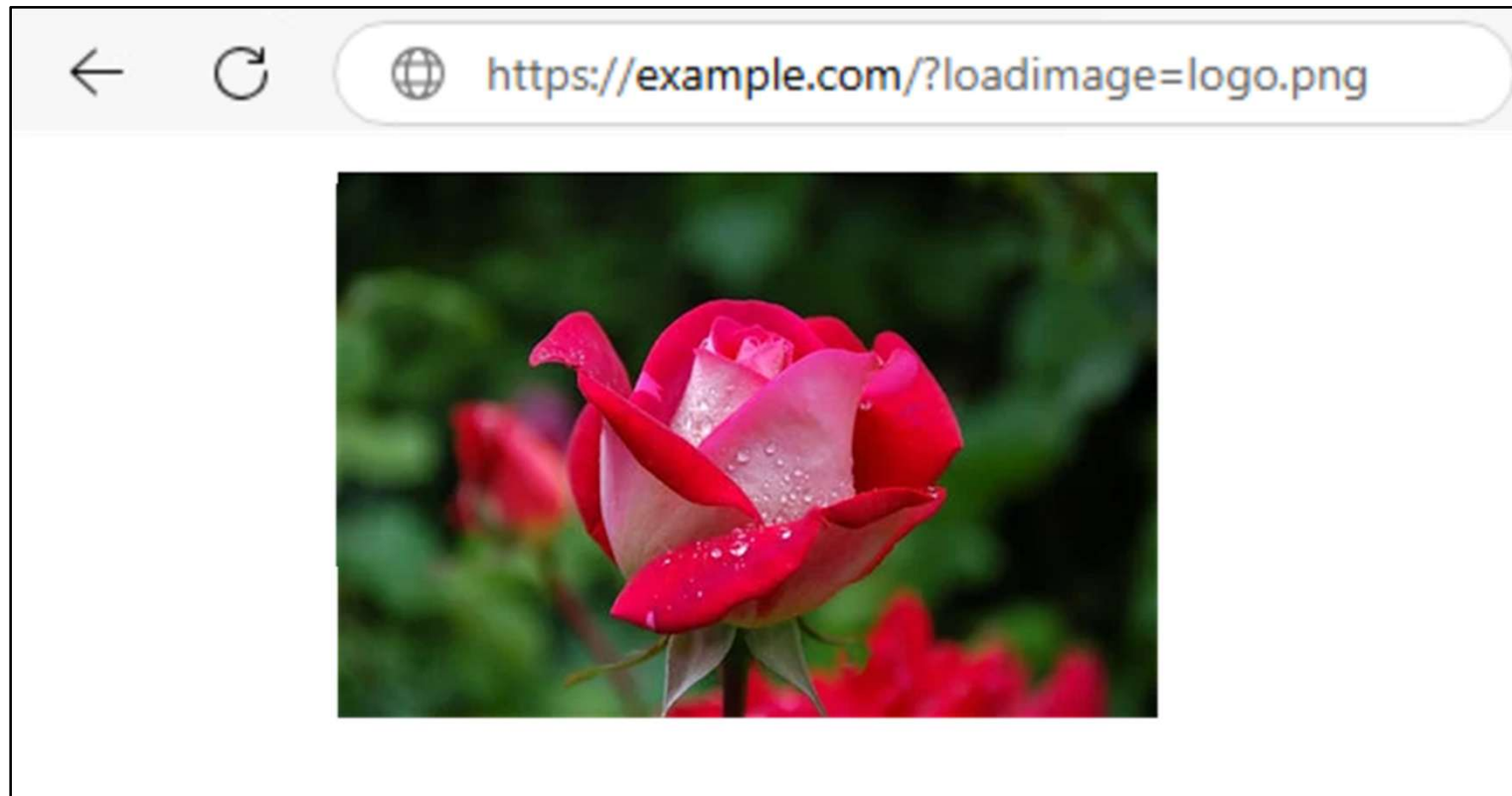
[Go to topic →](#)

2 Labs

Path\Directory Traversal

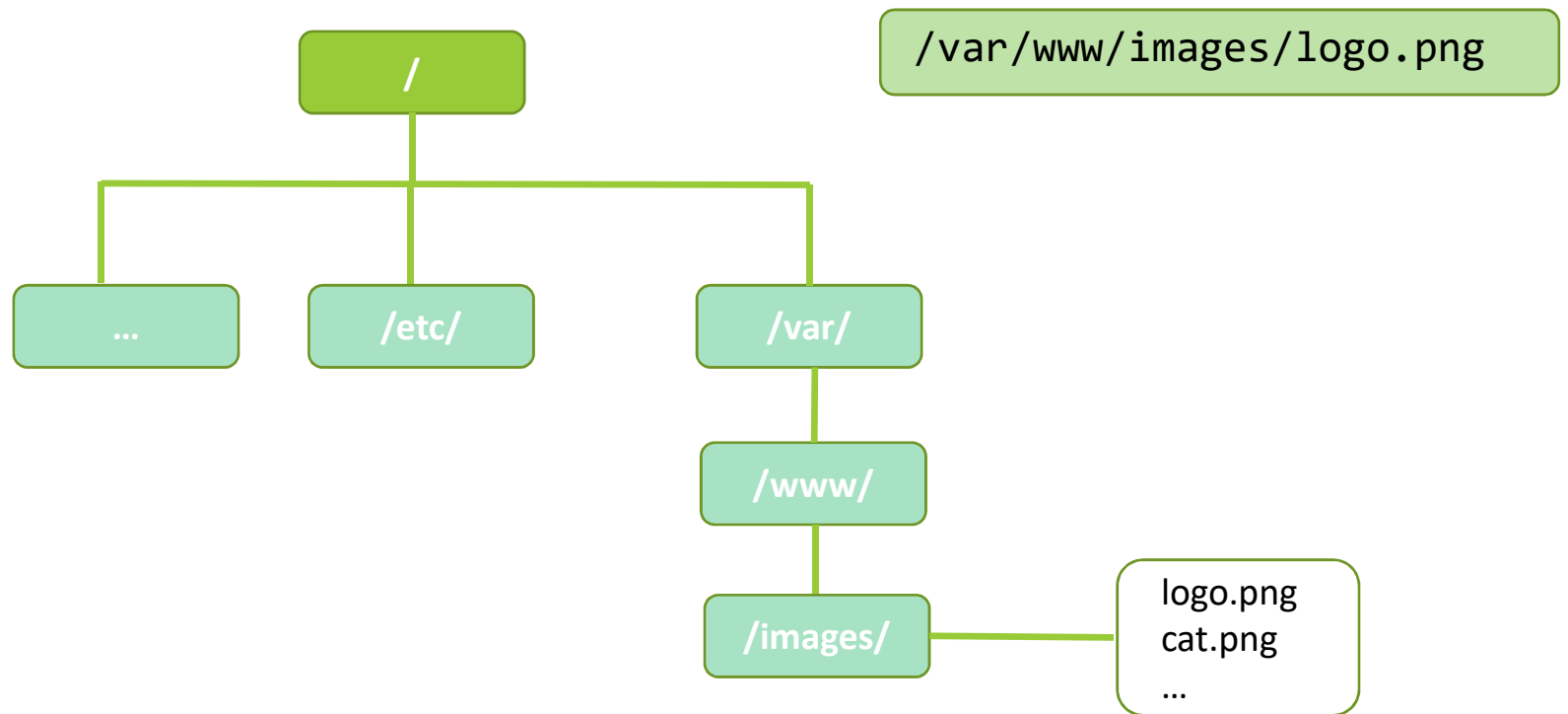


Path\Directory Traversal



Path\Directory Traversal

<https://example.com/?loadimage=logo.png>



Path\Directory Traversal

```
root@Ubuntu:~$ ls .  
/home/ubuntu
```



Current directory

```
root@Ubuntu:~$ ls ..  
/home
```



Parent directory

```
root@Ubuntu:~$ ls ../../..  
/
```



Two parent-directory

Path\Directory Traversal

Absolute Path

```
root@Ubuntu:~$ cat /etc/passwd  
root:x:0:0:root:/root:/bin/bash
```

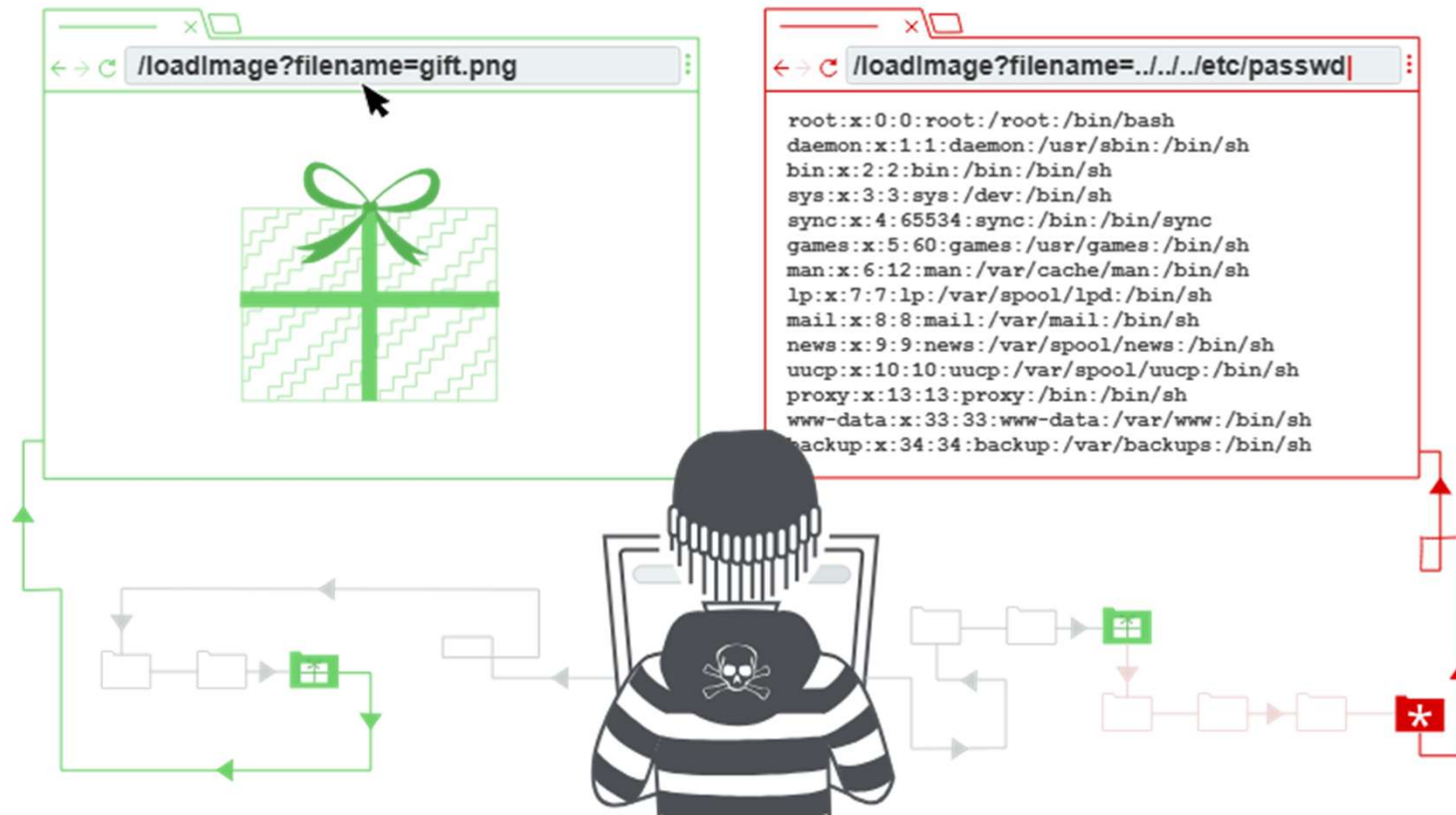
...

Relative Path

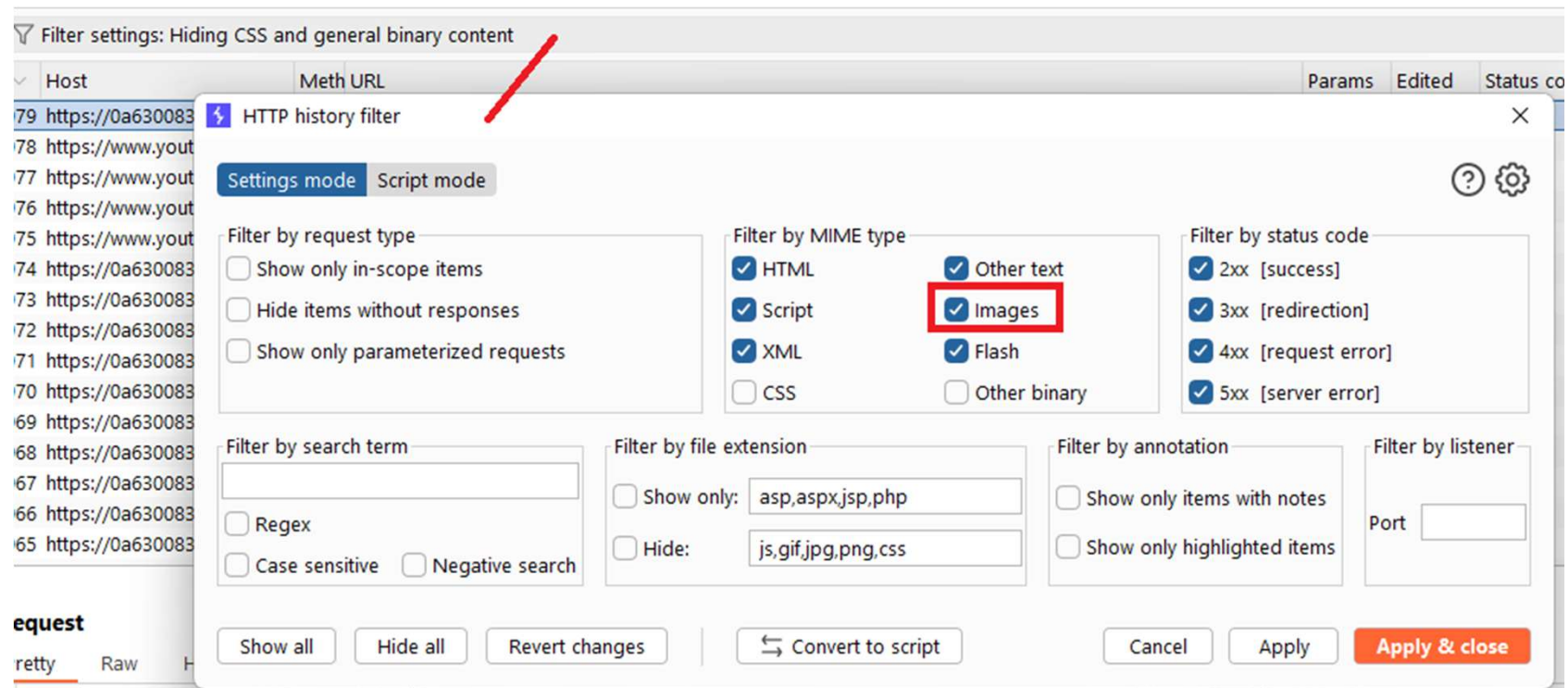
```
root@Ubuntu:~$ cat ../../etc/passwd  
root:x:0:0:root:/root:/bin/bash
```

...

Path\Directory Traversal



Path\Directory Traversal



Path\Directory Traversal - Lab

Lab Time - File path traversal, simple case

<https://portswigger.net/web-security/file-path-traversal/lab-simple>



Workshop Labs:

https://github.com/m2a2/Research/tree/main/Workshops/2025_Hackeriot

Path\Directory Traversal - Lab

Lab Time - File path traversal, traversal sequences blocked with absolute path bypass

<https://portswigger.net/web-security/file-path-traversal/lab-absolute-path-bypass>



Workshop Labs:

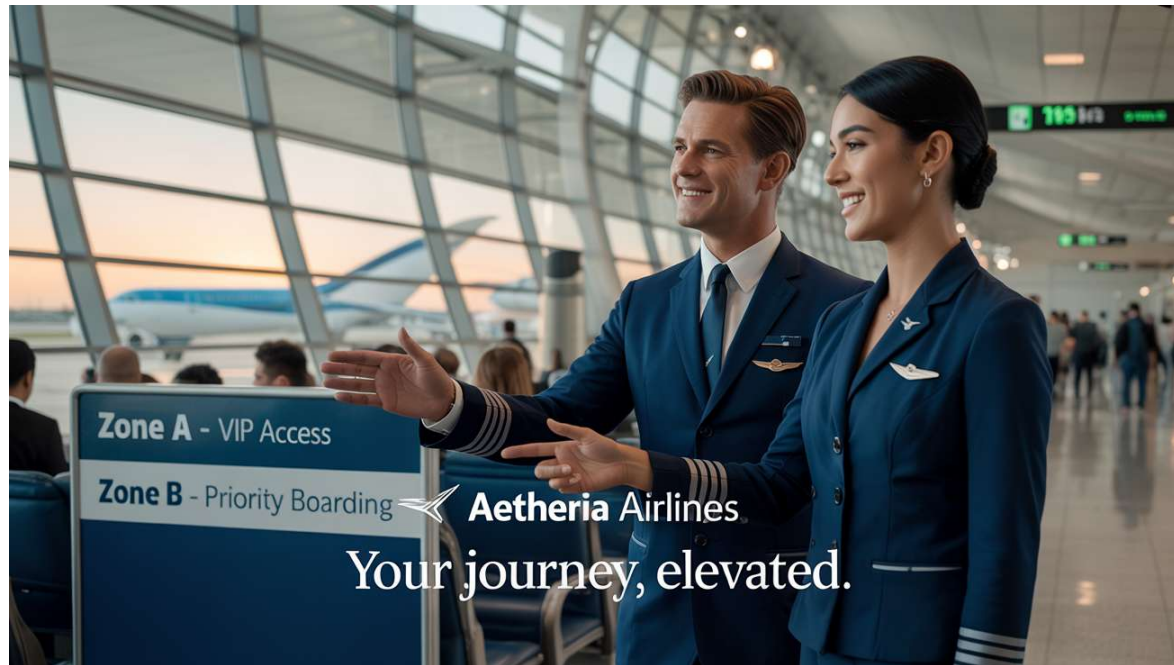
https://github.com/m2a2/Research/tree/main/Workshops/2025_Hackeriot

Access Control



Access Control

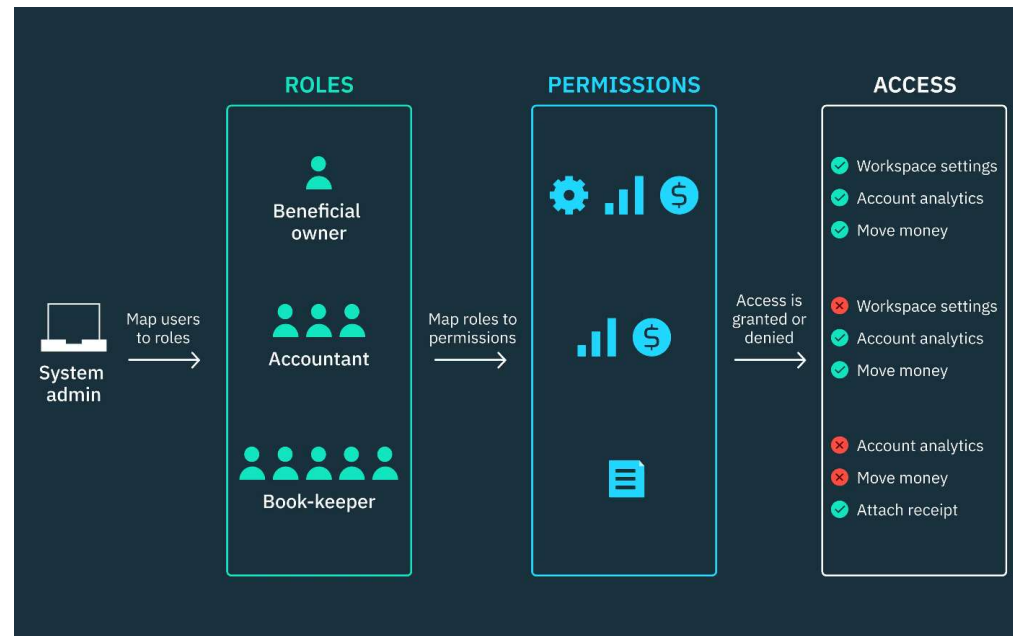
- מימוש הגבלות הקובעות מי רשאי/ת לגשת למשאבי המערכת או לבצע פעולות.



Generated by Ideogram

Access Controls Models

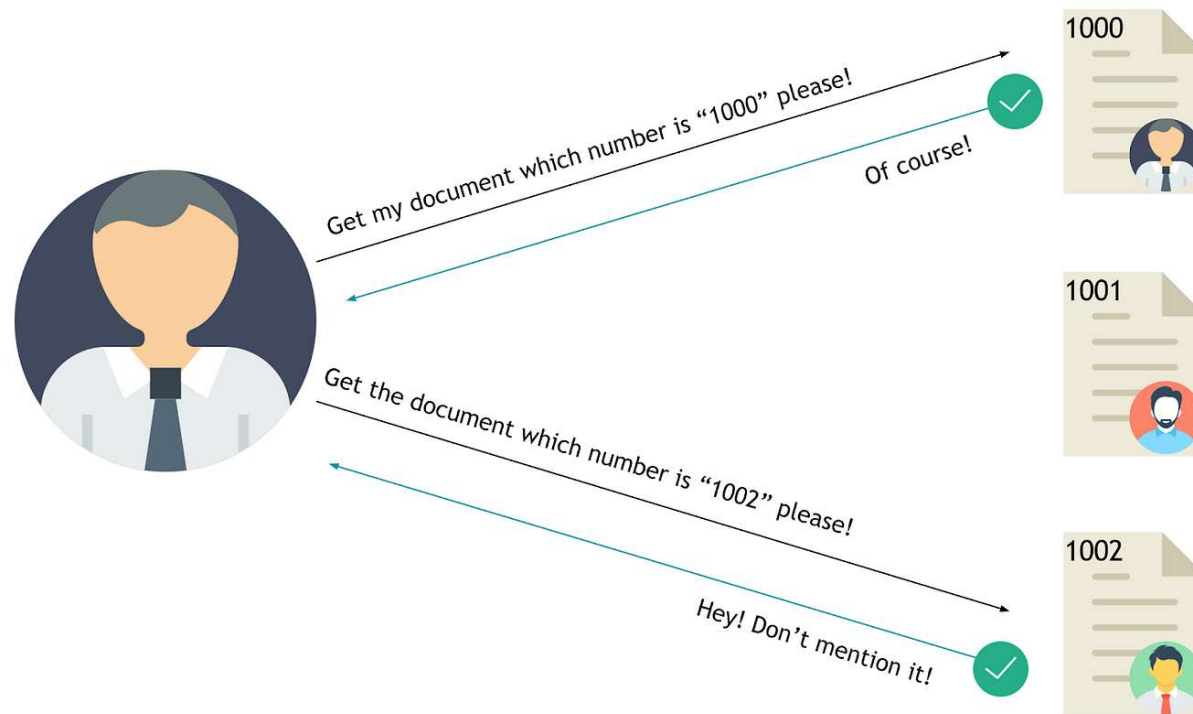
- ישנם מספר מודלים מופשטים שניתן ליישם לצורך Access control.
- Role Based Access Control (RBAC) הוא דוגמא נפוצה:



Exploiting Access Controls



Insecure Direct Object Reference (IDOR)



Access Control - Lab

Lab Time – Access Control

<https://portswigger.net/web-security/access-control/lab-unprotected-admin-functionality-with-unpredictable-url>



Workshop Labs:

https://github.com/m2a2/Research/tree/main/Workshops/2025_Hackeriot

Authentication



Login

Email:

Enter email

Password:

Enter password

☐ Show Password

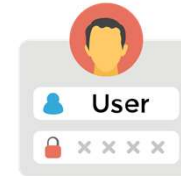
SIGN IN

Forgot [Username / Password?](#)

Don't have an account? [Sign up](#)

Authentication

Something you **know**



Something you **have**



Something you **are**

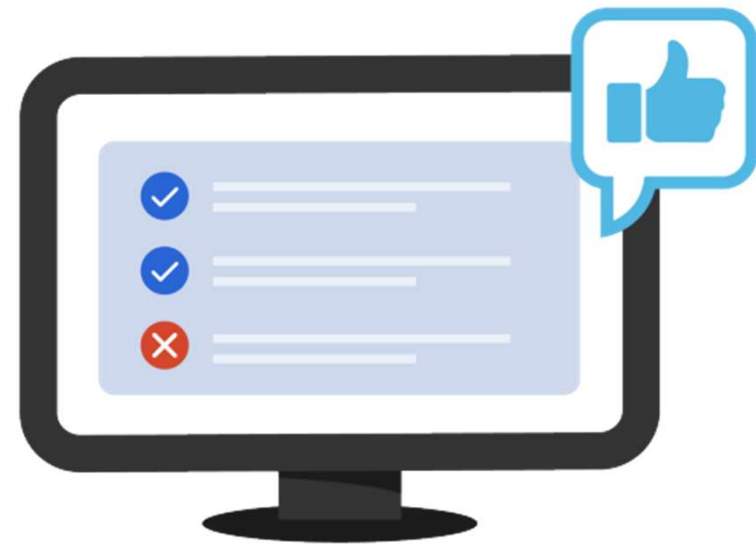


Authentication



Confirms users
are who they say they are.

Authorization



Gives users permission
to access a resource.

Authentication – Brute-force

Login

Email:

admin@example.com

Password:

☐ Show Password

SIGN IN



12345
123456
1234567
password
admin
111111111
mypassword

Authentication – Reset Password



Reset your password

Please enter your email address. We will send you an email to reset your password.



SEND EMAIL >

Authentication – Reset Password



User clicks 'Forgot Password'



<http://vulnerable-website.com/reset-password?user=victim-user>

New password

Confirm new password

Submit

Authentication – Reset Password

User clicks 'Forgot Password'



<http://vulnerable-website.com/reset-password?token=a0ba0d1cb3b63d13822572fcff1a241895d893f659164d4cc550b421ebdd48a8>

New password

Confirm new password

Submit

Authentication - Lab

Lab Time - Password reset broken logic

<https://portswigger.net/web-security/authentication/other-mechanisms/lab-password-reset-broken-logic>



Workshop Labs:

<https://github.com/m2a2/Research/tree/main/Workshops/2025> Hackeriot

Insecure Deserialization

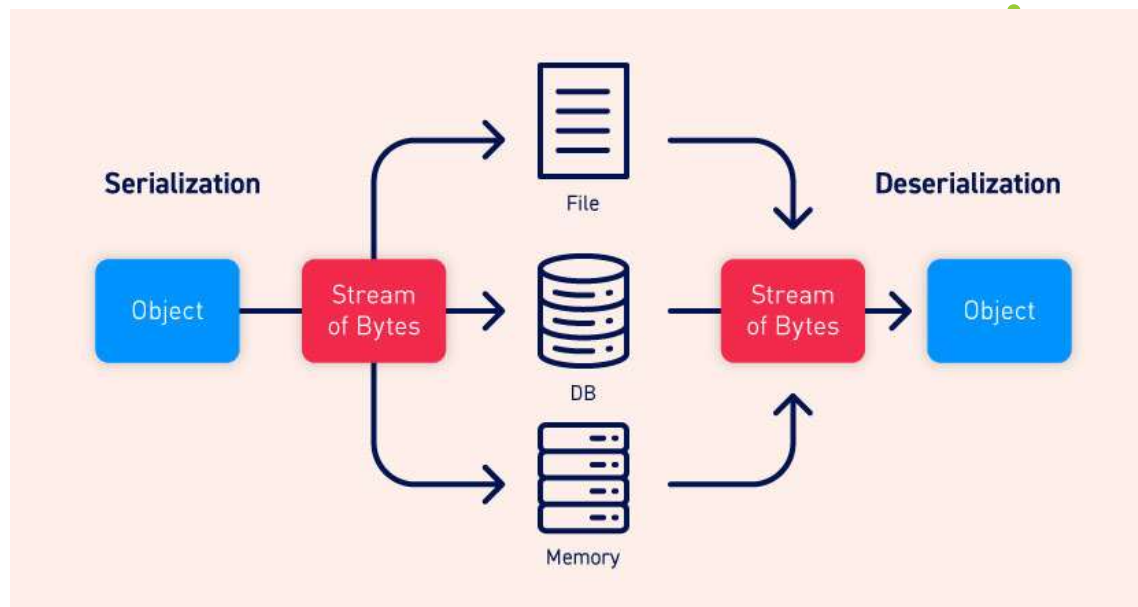
חולשה נפוצה – פתיחה לא מאובטחת של רצף סדרתי

Insecure Deserialization

- What is serialization?
- Serialization vs deserialization

• מהי סריאליזציה ?

• דיסריאליזציה



Insecure Deserialization - PHP serialize function

serialize

(PHP 4, PHP 5, PHP 7, PHP 8)

serialize — Generates a storable representation of a value

Description

```
serialize(mixed $value): string
```

Generates a storable representation of a value.

This is useful for storing or passing PHP values around without losing their type and structure.

To make the serialized string into a PHP value again, use [unserialize\(\)](#).

<https://www.php.net/manual/en/function.serialize.php>

Insecure Deserialization - PHP unserialize function

unserialize

(PHP 4, PHP 5, PHP 7, PHP 8)

unserialize — Creates a PHP value from a stored representation

Description

```
unserialize(string $data, array $options = []): mixed
```

`unserialize()` takes a single serialized variable and converts it back into a PHP value.

Warning Do not pass untrusted user input to `unserialize()` regardless of the `options` value of `allowed_classes`. Unserialization can result in code being loaded and executed due to object instantiation and autoloading, and a malicious user may be able to exploit this. Use a safe, standard data interchange format such as JSON (via `json_decode()` and `json_encode()`) if you need to pass serialized data to the user.

If you need to unserialize externally-stored serialized data, consider using `hash_hmac()` for data validation. Make sure data is not modified by anyone but you.

<https://www.php.net/manual/en/function.unserialize.php>

Demo: <https://www.programiz.com/php/online-compiler/>

Exploiting Insecure Deserialization Vulnerabilities

PHP serialization format

```
$user->name = "carlos";  
$user->isLoggedIn = true;
```

When serialized, this object may look something like this:

```
O:4:"User":2:{s:4:"name":s:6:"carlos"; s:10:"isLoggedIn":b:1;}
```

This can be interpreted as follows:

- O:4:"User" - An object with the 4-character class name "User"
- 2 - the object has 2 attributes
- s:4:"name" - The key of the first attribute is the 4-character string "name"
- s:6:"carlos" - The value of the first attribute is the 6-character string "carlos"
- s:10:"isLoggedIn" - The key of the second attribute is the 10-character string "isLoggedIn"
- b:1 - The value of the second attribute is the boolean value true

• פורמט הסריאליזציה של PHP

• פורמט מבוסס טקסט

• אותיות מייצגות את סוג המשתנה (Data Type)

• מספרים מייצגים אורך של השדה המקודד

<https://portswigger.net/web-security/deserialization/exploiting>

Insecure Deserialization

- What is insecure deserialization?

- מהי חולשת "פענוח לא בטוח" ?

- המידע לפיענוח נשלט ע"י המשתמש (או תוקף)

- אף ניתן לשחזר (לייצר) אובייקט מסוג אחר

Exploiting Insecure Deserialization Vulnerabilities

ניצול החולשה

Insecure Deserialization - Exploiting

- How to identify insecure deserialization

- כיצד נזהה חולשות "פענוח לא בטוח"

- סטטי (סקר קוד)

- במידה ויש לנו את קוד המקור

- נחפש את שמות הפונקציות הרלוונטיות לאותה השפה.

- דינאמי - (מבדק חדירות)

- במידה ומכירים את "פורמט הסריאליזציה" של אותה השפה

- ניתן לזהות ע"י הסתכלות על התוכן שמתקבל מהשרת.

- כאשר נזהה מידע כזה – ננסה לראות האם אנחנו יכולים לשלוט עליו

- [Deserialization - OWASP Cheat Sheet Series](#)

- <https://portswigger.net/web-security/deserialization/exploiting>

Exploiting Insecure Deserialization - Lab

Lab Time - Insecure Deserialization

<https://portswigger.net/web-security/deserialization/exploiting/lab-deserialization-modifying-serialized-objects>



Workshop Labs:

https://github.com/m2a2/Research/tree/main/Workshops/2025_Hackeriot

Insecure Deserialization

- How to identify insecure deserialization

- כיצד נזהה חולשות "פענוח לא בטוח"

- סטטי (סקר קוד)

- במידה ויש לנו את קוד המקור

- נחפש את שמות הפונקציות הרלוונטיות לאותה השפה.

- דינאמי - (מבדק חדירות)

- במידה ומכירים את "פורמט הסריאליזציה" של אותה השפה

- ניתן לזהות ע"י הסתכלות על התוכן שמתקבל מהשרת.

- כאשר נזהה מידע כזה – ננסה לראות האם אנחנו יכולים לשלוט עליו

- [Deserialization - OWASP Cheat Sheet Series](#)

- <https://portswigger.net/web-security/deserialization/exploiting>

הגנות - Insecure Deserialization

- How to prevent insecure deserialization vulnerabilities ?
 - כיצד נמנע מתקפות "פענוח לא בטוח" ?
 - להימנע ככל הניתן מדיסריאליזציה של תוכן (שנשלט ע"י משתמש)
 - יישום בדיקות אמינות (Integrity)
 - אכיפת מגבלות סוג אובייקט (מחמירות) בתהליך פתיחה של רצף סדרתי
 - בידוד והרצת קוד לפתיחת רצף סדרתי בסביבה בעלת הרשאות נמוכות ככל הניתן
- <https://owasp.org/www-pdf-archive/OWASP-Top-10-2017-he.pdf>

Insecure Deserialization

- OWASP Top 10 Reference:
 - https://owasp.org/www-project-top-ten/2017/A8_2017-Insecure_Deserialization
- OWASP Deserialization Cheat Sheet
 - https://cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.html
- Portswigger - Web Security Academy - Insecure deserialization
 - <https://portswigger.net/web-security/deserialization>

Insecure Deserialization – Real World Example

CVE's



Insecure Deserialization – Real World – More CVE's

- <https://cve.mitre.org/cgi-bin/cvekey.cgi?keyword=deserialization>
- <https://www.cvedetails.com/google-search-results.php?q=deserialization>
- <https://www.cvedetails.com/vulnerability-list/cwe-502/vulnerabilities.html>

Resources & Next Steps

Practice, Socialize, To the champions league

Agenda

- רקע תיאורטי
- כלים לפיתוח ומחקר ווב
- אבטחת מערכות ווב – (Web Security)
 - חולשות נפוצות + מעבדה עצמית
 - Path Traversal
 - Access Control
 - Authentication
 - Insecure deserialization
- מקורות תרגול והעשרה + סיכום

תרגול עצמי – Practice – Web Security

- **Pico-CTF
 - <https://www.picoctf.org/>
- Overthewire – Bandit + **Natas
 - <https://overthewire.org/wargames/>
- **Portswigger - Web Security Academy - Labs
 - <https://portswigger.net/web-security/all-labs>
 - <https://portswigger.net/web-security/learning-path>
- OWASP – Practice Systems
 - <https://github.com/digininja/DVWA>
 - <https://github.com/juice-shop/juice-shop>
 - <https://github.com/WebGoat/WebGoat>
- HackTheBox (more system-related, also web challenges)
 - <https://www.hackthebox.eu/>

Web Security – Social

- OWASP – Meetup & AppSecIL
 - <https://www.meetup.com/OWASP-Israel/>
- BSidesTLV – (11.2025)
 - <https://bsidestlv.com/>
 - <https://bsidestlv.com/ctf/>
- Intent Summit (CyberArk) – (25.11.2025)
 - <https://intentsummit.org/>
- BlueHatIL (Microsoft)
 - <https://www.microsoftrnd.co.il/BluehatIL/home>
- DefCon
 - <https://defcon.org/index.html>
 - <https://www.youtube.com/user/DEFCONConference>
- BlackHat
 - <https://www.blackhat.com/>
 - <https://www.youtube.com/user/BlackHatOfficialYT>

• התנדבות?
• התנדבות!

• הקלטות?
• הקלטות!

תחרויות ותוכניות חיפוש חולשות - CTF & Bug Bounty

- CTF (My two cents)
 - Try Alone
 - Write down what you tried
 - Then (after the event), Read Write-ups!
- CTF Time
 - <https://ctftime.org/event/list/upcoming>
 - לוח שנה עם תחרויות קרובות
- Bug Bounty - Platforms
 - <https://hackerone.com/bug-bounty-programs>
 - <https://www.bugcrowd.com/bug-bounty-list/>
 - <https://www.intigriti.com/>
- Bug Bounty – Programs for specific companies
 - <https://www.guru99.com/bug-bounty-programs.html>

Summary

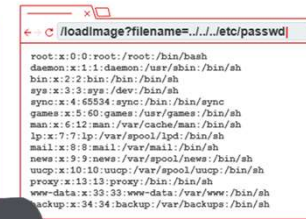
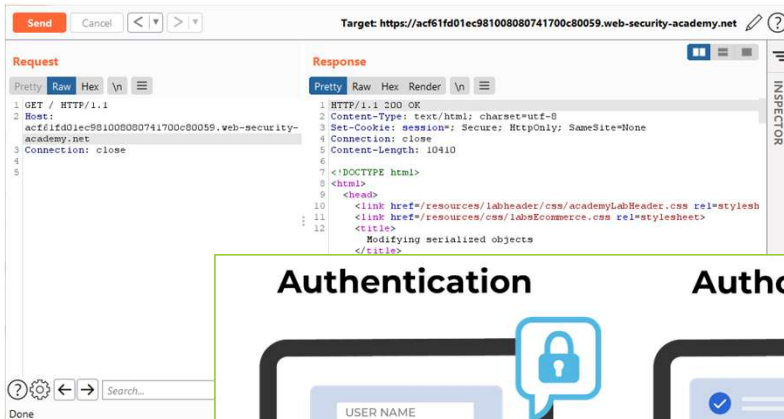


Feedback

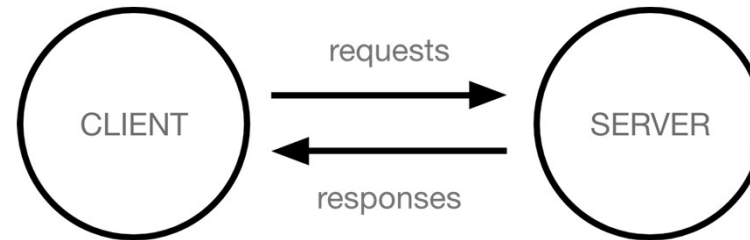


נותרו עוד שני שקפים...

סיכום



- רקע תיאורטי
- פרוטוקול – HTTP
- כלים שימושיים
- אבטחת ווב
- Path Traversal
- Access Control (Authorization)
- הזדהות לעומת בקרת גישה
- מקורות להעשרה ותרגול עצמי



Questions ? (&Thank You!)

Contact Us



[Maor Abutbul]
<https://il.linkedin.com/in/maor-abutbul>



[Eviatar Gerzi]
<https://il.linkedin.com/in/eviatar-g-25bb6225>



[Lior Yakim]
<https://il.linkedin.com/in/lior-yakim-79b100156>

